

Heaven's Light is Our Guide
Rajshahi University of Engineering and Technology



Course Code

MTE 4118

Course Title

Control System and Robotics Sessional

Experiment Date: October 15, 2025,

Submission Date: October 29, 2025

Lab Report 5:

Root Locus and Time Response Analysis of DC Motor under PID, PI, and PD Control

Submitted to

Md. Sakib Hassan Chowdhury

Lecturer

Dept of MTE, RUET

Submitted by

Md. Tajim An Noor

Roll: 2010025

Contents

Objectives	1
Theory	1
Overview	1
DC motor model	1
Controller structures	1
Qualitative effects	2
Closed-loop form	2
Design criteria and motor data	2
Time-response metrics	2
Tools Used	3
Working Principle	3
Code	3
K_P, K_I, K_D Gains	4
Output	4
Code Output	5
Discussion	5
Conclusion	5
References	5

Experiment 5

Root Locus and Time Response Analysis of DC Motor under PID, PI, and PD Control

Objectives

- To analyze the root locus of a DC motor control system.
- To implement and compare PID, PI, and PD controllers for speed regulation.
- To evaluate system performance through time response analysis using MATLAB Simulink.

Theory

Overview

This experiment examines root-locus and time-domain behavior of an armature-controlled DC motor regulated by P, PI, PD, and PID controllers. Analysis uses the motor's electromechanical model, classical controller forms, and closed-loop pole placement via root-locus [1, 2].

DC motor model

The armature-controlled motor couples electrical and mechanical dynamics:

$$L \frac{di_a(t)}{dt} + Ri_a(t) + K_e \omega(t) = v_a(t), \quad J \frac{d\omega(t)}{dt} + B\omega(t) = K_t i_a(t),$$

where v_a is armature voltage, i_a armature current, ω angular speed, and R, L, J, B, K_e, K_t are motor constants. Taking Laplace transforms and eliminating $I_a(s)$ yields the transfer function from armature voltage to speed:

$$G(s) \equiv \frac{\Omega(s)}{V_a(s)} = \frac{K_t}{(Js + B)(Ls + R) + K_e K_t},$$

which follows standard electromechanical modelling procedures [3, 1].

Controller structures

Standard linear controllers used are:

$$C_{PI}(s) = K_p + \frac{K_i}{s} = \frac{K_p(s + \frac{K_i}{K_p})}{s},$$

$$C_{PD}(s) = K_p + K_d s = K_d \left(s + \frac{K_p}{K_d} \right).$$

$$C_{PID}(s) = K_p + \frac{K_i}{s} + K_d s = \frac{K_d \left(s^2 + s \left(\frac{K_p}{K_d} \right) + \frac{K_i}{K_d} \right)}{s}.$$

These controller forms and their tuning implications are discussed in classical texts and PID literature [2, 4].

Qualitative effects of actions

1. Proportional (P): raises gain — faster response, reduces but doesn't eliminate steady-state error for type-0 systems.
2. Integral (I): adds a pole at 0 — eliminates step steady-state error, at expense of phase margin.
3. Derivative (D): adds phase lead — improves damping and lowers overshoot, but amplifies high-frequency noise.

Closed-loop form and design approach

For unity feedback the closed-loop transfer from reference $R(s)$ to output $\Omega(s)$ is

$$T(s) = \frac{C(s)G(s)}{1 + C(s)G(s)},$$

with characteristic equation $1 + C(s)G(s) = 0$. Design proceeds by inspecting the root locus of $G(s)$ (and the effect of controller zeros/poles) and selecting controller gains so closed-loop poles meet desired transient specifications [1, 2].

Design criteria and motor data

Motor parameters used in this lab:

$$J = 0.01, \quad B = 0.1, \quad K_e = K_t = 0.01, \quad R = 1, \quad L = 0.5.$$

Performance targets: settling time $T_s \approx 1$ s (2% criterion), and maximum overshoot $M_p < 5\%$ [5].

Time-response metrics

- Rise time t_r : time to go from 10% to 90% of final value.
- Peak time t_p : time to the first maximum peak.
- Settling time T_s : time to remain within a 2% (or 5%) band.
- Percent overshoot M_p : $\frac{\text{peak} - \text{steady}}{\text{steady}} \times 100\%$.
- Steady-state error e_{ss} : final deviation from reference (reduced by integral action).

Tools Used

- MATLAB
- $\text{\LaTeX}$

Working Principle

Code

```
1  clc;
2  clear;
3  close all;
4
5  % --- DC motor parameters (from
   ↪   handout) ---
6  J = 0.01; % kg*m^2
7  b = 0.1; % N*m*s
8  K = 0.01; % torque/back-EMF constant
9  R = 1; % Ohm
10 L = 0.5; % H
11
12 s = tf('s');
13 TF = K / ((J*s + b)*(L*s + R) + K^2); %
   ↪   Voltage -> angular velocity
14
15 % === Controller gains (replace with
   ↪   your tuned values) ===
16 % PI gains
17 Kp_PI = 23.665;
18 Ki_PI = 43.66192;
19 Kd_PI = 0;
20
21 % PD gains
22 Kp_PD = 530.43375;
23 Ki_PD = 0;
24 Kd_PD = 91.85;
25
26 % PID gains
27 Kp_PID = 594.048;
28 Ki_PID = 1971.592;
29 Kd_PID = 47.6;
30
31 % Controllers
32 C_PI = pid(Kp_PI, Ki_PI, Kd_PI);
33 C_PD = pid(Kp_PD, Ki_PD, Kd_PD);
34 C_PID = pid(Kp_PID, Ki_PID, Kd_PID);
35
36 % Closed-loop systems (unity feedback)
37 CL_PI = feedback(C_PI*TF, 1);
38 CL_PD = feedback(C_PD*TF, 1);
39 CL_PID = feedback(C_PID*TF, 1);
40
41 % --- Figure: Step responses (separate
   ↪   figures) ---
42 tfinal = 3; % seconds
43
44 figure(1);
45 step(CL_PI, tfinal);
46 grid on;
47 title('Step Response - PI');
48
49 figure(2);
50 step(CL_PD, tfinal);
51 grid on;
52 title('Step Response - PD');
53
54 figure(3);
55 step(CL_PID, tfinal);
56 grid on;
57 title('Step Response - PID');
58
59 % --- Root loci (separate figures) ---
60 figure(4);
61 rlocus(C_PI*TF);
62 grid on;
63 title('Root Locus - PI');
64
65 figure(5);
66 rlocus(C_PD*TF);
67 grid on;
68 title('Root Locus - PD');
69
70 figure(6);
71 rlocus(C_PID*TF);
72 grid on;
73 title('Root Locus - PID');
```

```

75 % --- Performance numbers ---
76 S_PI = stepinfo(CL_PI);
77 S_PD = stepinfo(CL_PD);
78 S_PID = stepinfo(CL_PID);
79
80 disp('Step info - PI:');
81 disp(S_PI);
82 disp('Step info - PD:');
83 disp(S_PD);
84 disp('Step info - PID:');
85 disp(S_PID);

```

K_P, K_I, K_D Gains

Controller	K_p	K_i	K_d
PI	23.665	43.66192	0
PD	530.43375	0	91.85
PID	594.048	1971.592	47.6

Table 1: Controller gains

Output

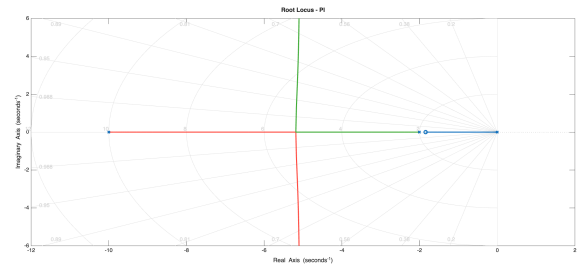
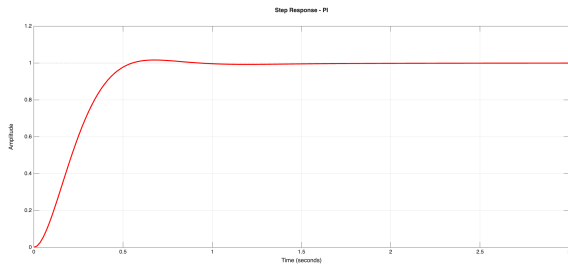


Figure 1: PI Controller: (Left) Step Response, (Right) Root Locus

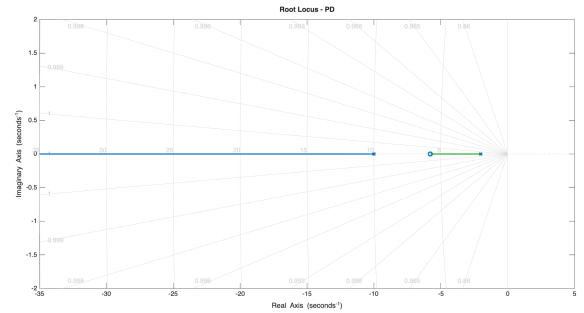
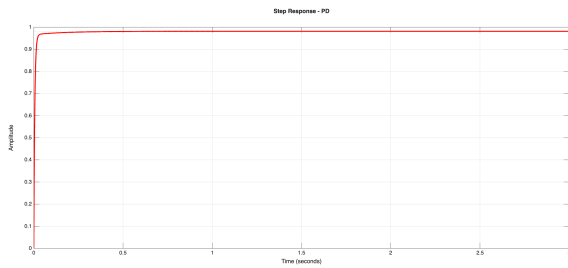


Figure 2: PD Controller: (Left) Step Response, (Right) Root Locus

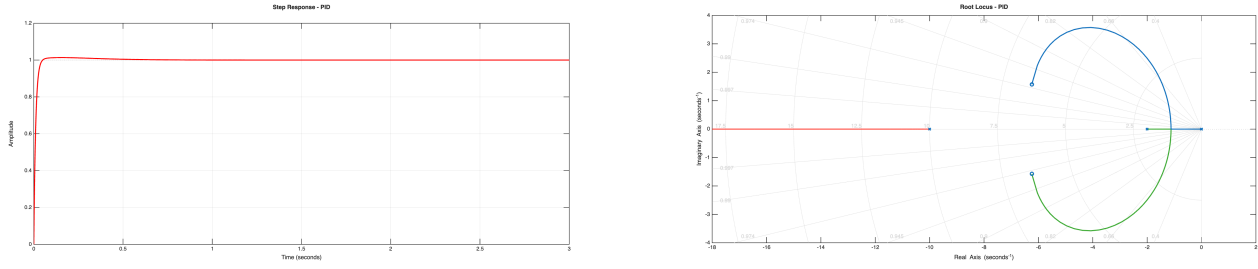


Figure 3: PID Controller: (Left) Step Response, (Right) Root Locus

Code Output

Metric	PI	PD	PID
Rise time (s)	0.3343	0.0123	0.0226
Transient time (s)	0.5039	0.0263	0.0380
Settling time (s)	0.5039	0.0263	0.0380
Settling min	0.9073	0.8870	0.9052
Settling max	1.0166	0.9764	1.0134
Overshoot (%)	1.6559	0.0000	1.3382
Undershoot (%)	0.0000	0.0000	0.0000
Peak	1.0166	0.9764	1.0134
Peak time (s)	0.6756	0.1936	0.1543

Table 2: Step-response metrics for PI, PD, and PID controllers

Discussion

Root-locus guided controller tuning produced the expected trade-offs between speed and accuracy. Key observations:

- PI: removed steady-state error; $T_s \approx 0.50\text{s}$, $M_p \approx 1.66\%$.
- PD: fastest transients but residual offset; $T_s \approx 0.03\text{s}$, $final \approx 0.976$.
- PID: balanced performance—fast settling, low overshoot, near-zero steady-state error; $T_s \approx 0.04\text{s}$, $M_p \approx 1.34\%$.

Recommendation: add a low-pass filter on the derivative path and evaluate robustness to parameter variations.

Conclusion

Design objectives (overshoot $\leq 5\%$ and acceptable settling) were met by the tuned controllers: the PI controller effectively eliminated steady-state error while maintaining low overshoot and a short settling time; the PD controller accelerated the transient response and reduced overshoot but exhibited a residual steady-state error due to lack of integral action; and the PID controller provided a balanced compromise with fast settling, low overshoot, and minimal steady-state error. Recommendations: implement derivative action with a practical low-pass filter to limit noise amplification and evaluate robustness to plant uncertainty and parameter variation in future work.

References

- [1] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [2] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 7th ed. Boston, MA: Pearson, 2014.
- [3] N. S. Nise, *Control Systems Engineering*, 7th ed. Hoboken, NJ: Wiley, 2015.
- [4] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC: Instrument Society of America, 1995.
- [5] The MathWorks, Inc., *MATLAB and Simulink*, Natick, MA, 2020. [Online]. Available: <https://www.mathworks.com>